

Universidad ORT Uruguay
Facultad de Ingeniería
Escuela de Tecnología

OBLIGATORIO PROGRAMACIÓN 2



Franco Beloqui – 360021



Emiliano Baston- 163914

N2E

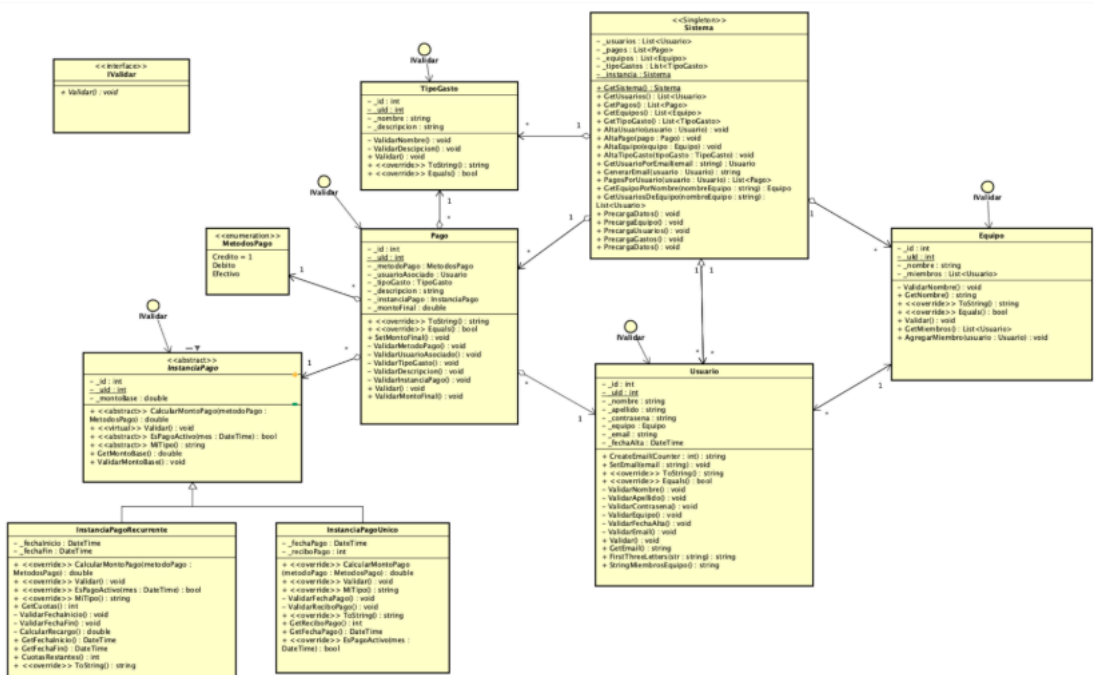
Docente: ANDRÉS URETA

Analista Programador

Fecha de entrega del documento: 16-10-2025

1. Diagrama de clases del Dominio	3
2. Tabla de datos precargados	4
2.1 Precarga Equipos	4
2.2 Precarga Usuarios	5
2.3 Precarga Gastos	9
2.4 Precarga Pagos	9
3. Código fuente	12
3.1 Clase Sistema	12
3.2 Clase Usuario	24
3.3 Clase Equipo	27
3.4 Clase Tipo de Gasto	29
3.5 Clase Pago	30
3.6 Clase Instancia Pago (Abstracta)	33
3.7 Clase Pago Recurrente	35
3.8 Clase Pago Único	38
3.9 Método Pago (Enumeration)	40
3.10 IValidar (Interface)	40
3.11 Program	41
4. Prompts AI	48

1. Diagrama de clases del Dominio



2. Tabla de datos precargados

2.1 Precarga Equipos

```
public void PrecargaEquipo()  
{  
    Equipo contabilidad = new Equipo("Contabilidad");  
    AltaEquipo(contabilidad);  
  
    Equipo finanzas = new Equipo("Finanzas");  
    AltaEquipo(finanzas);  
  
    Equipo tesoreria = new Equipo("Tesoreria");  
    AltaEquipo(tesoreria);  
  
    Equipo auditoria = new Equipo("Auditoria");  
    AltaEquipo(auditoria);  
  
    Equipo ventas = new Equipo("Ventas");  
    AltaEquipo(ventas);  
}
```

2.2 Precarga Usuarios

```
public void PrecargaUsuarios()
{
    // Equipos existentes

    Equipo contabilidad = GetEquipoPorNombre("Contabilidad");

    Equipo finanzas    = GetEquipoPorNombre("Finanzas");

    Equipo tesoreria   = GetEquipoPorNombre("Tesoreria");

    Equipo auditoria   = GetEquipoPorNombre("Auditoria");


    // ---- Contabilidad (6) ----

    Usuario usuario1 = new Usuario("Juana", "Melcer", "12345678", contabilidad);

    AltaUsuario(usuario1);

    contabilidad.AgregarMiembro(usuario1);


    Usuario usuario2 = new Usuario("Ana", "Perez", "12345678", contabilidad);

    AltaUsuario(usuario2);

    contabilidad.AgregarMiembro(usuario2);


    Usuario usuario3 = new Usuario("Lucas", "Garcia", "12345678", contabilidad);

    AltaUsuario(usuario3);

    contabilidad.AgregarMiembro(usuario3);


    Usuario usuario4 = new Usuario("Martina", "Rodriguez", "12345678", contabilidad);

    AltaUsuario(usuario4);

    contabilidad.AgregarMiembro(usuario4);
```

```
    Usuario usuario5 = new Usuario("Santiago", "Fernandez", "12345678",  
contabilidad);
```

```
    AltaUsuario(usuario5);
```

```
    contabilidad.AgregarMiembro(usuario5);
```

```
    Usuario usuario6 = new Usuario("Valentina", "Gomez", "12345678", contabilidad);
```

```
    AltaUsuario(usuario6);
```

```
    contabilidad.AgregarMiembro(usuario6);
```

```
// ---- Finanzas (5) ----
```

```
    Usuario usuario7 = new Usuario("Mateo", "Suarez", "12345678", finanzas);
```

```
    AltaUsuario(usuario7);
```

```
    finanzas.AgregarMiembro(usuario7);
```

```
    Usuario usuario8 = new Usuario("Camila", "Lopez", "12345678", finanzas);
```

```
    AltaUsuario(usuario8);
```

```
    finanzas.AgregarMiembro(usuario8);
```

```
    Usuario usuario9 = new Usuario("Joaquin", "Alvarez", "12345678", finanzas);
```

```
    AltaUsuario(usuario9);
```

```
    finanzas.AgregarMiembro(usuario9);
```

```
    Usuario usuario10 = new Usuario("Sofia", "Martinez", "12345678", finanzas);
```

```
    AltaUsuario(usuario10);
```

```
    finanzas.AgregarMiembro(usuario10);
```

```
    Usuario usuario11 = new Usuario("Tomas", "Pereira", "12345678", finanzas);
```

```
AltaUsuario(usuario11);

finanzas.AgregarMiembro(usuario11);

// ---- Tesoreria (5) ----

Usuario usuario12 = new Usuario("Isabella", "Santos", "12345678", tesoreria);
AltaUsuario(usuario12);
tesoreria.AgregarMiembro(usuario12);

Usuario usuario13 = new Usuario("Agustin", "Cabrera", "12345678", tesoreria);
AltaUsuario(usuario13);
tesoreria.AgregarMiembro(usuario13);

Usuario usuario14 = new Usuario("Mia", "Torres", "12345678", tesoreria);
AltaUsuario(usuario14);
tesoreria.AgregarMiembro(usuario14);

Usuario usuario15 = new Usuario("Facundo", "Silva", "12345678", tesoreria);
AltaUsuario(usuario15);
tesoreria.AgregarMiembro(usuario15);

Usuario usuario16 = new Usuario("Julia", "Ramos", "12345678", tesoreria);
AltaUsuario(usuario16);
tesoreria.AgregarMiembro(usuario16);

// ---- Auditoria (5) ----

Usuario usuario17 = new Usuario("Nicolas", "Castro", "12345678", auditoria);
```

```
AltaUsuario(usuario17);

auditoria.AgregarMiembro(usuario17);


Usuario usuario18 = new Usuario("Florence", "Vega", "12345678", auditoria);

AltaUsuario(usuario18);

auditoria.AgregarMiembro(usuario18);


Usuario usuario19 = new Usuario("Andres", "Morales", "12345678", auditoria);

AltaUsuario(usuario19);

auditoria.AgregarMiembro(usuario19);


Usuario usuario20 = new Usuario("Carolina", "Ruiz", "12345678", auditoria);

AltaUsuario(usuario20);

auditoria.AgregarMiembro(usuario20);


Usuario usuario21 = new Usuario("Bruno", "Herrera", "12345678", auditoria);

AltaUsuario(usuario21);

auditoria.AgregarMiembro(usuario21);


Usuario usuario22 = new Usuario("Lucia", "Acosta", "12345678", auditoria);

AltaUsuario(usuario22);

    auditoria.AgregarMiembro(usuario22); }
```


2.3 Precarga Gastos

```
public void PrecargaGastos()
{
    TipoGasto tipo1 = new TipoGasto("Alquiler", "Pago mensual por el
alquiler de la oficina");
    AltaTipoGasto(tipo1);
    TipoGasto tipo2 = new TipoGasto("Servicios", "Pago de servicios como
luz, agua, internet, etc.");
    AltaTipoGasto(tipo2);
    TipoGasto tipo3 = new TipoGasto("Sueldos", "Pago de sueldos a los
empleados");
    AltaTipoGasto(tipo3);
    TipoGasto tipo4 = new TipoGasto("Materiales de oficina", "Compra de
materiales de oficina como papel, bolígrafos, etc.");
    AltaTipoGasto(tipo4);
    TipoGasto tipo5 = new TipoGasto("Publicidad", "Gastos en publicidad y
marketing");
    AltaTipoGasto(tipo5);
    TipoGasto tipo6 = new TipoGasto("Viajes", "Gastos en viajes de
negocios");
    AltaTipoGasto(tipo6);
}
```

2.4 Precarga Pagos

```
public void PrecargaPagos()
{
    // ===== 6 pagos RECURRENTES SIN fecha de fin (EndDate = null)
    =====
    AltaPago(new Pago(MetodosPago.Credito, Usuarios[0],
TipoGastos[0], "Alquiler mensual",new
InstanciaPagoRecurrente(28000, DateTime.Now.AddMonths(-12),
null)));
    AltaPago(new Pago(MetodosPago.Debito, Usuarios[1],
TipoGastos[1], "Servicios (UTE/ANTEL)",new
InstanciaPagoRecurrente(6500, DateTime.Now.AddMonths(-9),
null)));
    AltaPago(new Pago(MetodosPago.Credito, Usuarios[2],
TipoGastos[2], "Sueldos",new
InstanciaPagoRecurrente(120000,DateTime.Now.AddMonths(-10),
null)));
    AltaPago(new Pago(MetodosPago.Credito, Usuarios[3],
TipoGastos[3], "Mantenimiento",new InstanciaPagoRecurrente(4200,
DateTime.Now.AddMonths(-6), null)));
}
```

```

    AltaPago(new Pago(MetodosPago.Debito, Usuarios[4],
TipoGastos[4], "Internet",new InstanciaPagoRecurrente(1800,
DateTime.Now.AddMonths(-8), null)));

    AltaPago(new Pago(MetodosPago.Credito, Usuarios[5],
TipoGastos[5], "Limpieza",new InstanciaPagoRecurrente(3500,
DateTime.Now.AddMonths(-5), null)));

    // ===== 9 pagos RECURRENTES con fecha de fin en el FUTURO =====
    AltaPago(new Pago(MetodosPago.Credito, Usuarios[6],
TipoGastos[0], "Licencia software",new
InstanciaPagoRecurrente(2200, DateTime.Now.AddMonths(-4),
DateTime.Now.AddMonths(8))));
    AltaPago(new Pago(MetodosPago.Debito, Usuarios[7],
TipoGastos[1], "Seguro flota",new InstanciaPagoRecurrente(9500,
DateTime.Now.AddMonths(-7), DateTime.Now.AddMonths(5))));
    AltaPago(new Pago(MetodosPago.Credito, Usuarios[8],
TipoGastos[2], "Alquiler depósito",new
InstanciaPagoRecurrente(54000, DateTime.Now.AddMonths(-2),
DateTime.Now.AddMonths(10))));
    AltaPago(new Pago(MetodosPago.Credito, Usuarios[9],
TipoGastos[3], "Servicio guardia",new
InstanciaPagoRecurrente(4100, DateTime.Now.AddMonths(-3),
DateTime.Now.AddMonths(4))));
    AltaPago(new Pago(MetodosPago.Debito, Usuarios[10],
TipoGastos[4], "Telefonía móvil",new
InstanciaPagoRecurrente(2600, DateTime.Now.AddMonths(-1),
DateTime.Now.AddMonths(12))));
    AltaPago(new Pago(MetodosPago.Credito, Usuarios[11],
TipoGastos[5], "Mantenimiento HVAC",new
InstanciaPagoRecurrente(7800, DateTime.Now.AddMonths(-6),
DateTime.Now.AddMonths(2))));
    AltaPago(new Pago(MetodosPago.Credito, Usuarios[12],
TipoGastos[0], "Soporte técnico",new
InstanciaPagoRecurrente(3300, DateTime.Now.AddMonths(-5),
DateTime.Now.AddMonths(6))));
    AltaPago(new Pago(MetodosPago.Debito, Usuarios[13],
TipoGastos[1], "Licencias antivirus",new
InstanciaPagoRecurrente(1450, DateTime.Now.AddMonths(-8),
DateTime.Now.AddMonths(3))));
    AltaPago(new Pago(MetodosPago.Credito, Usuarios[14],
TipoGastos[2], "Estacionamiento",new
InstanciaPagoRecurrente(2100, DateTime.Now.AddMonths(-2),
DateTime.Now.AddMonths(7))));

    // ===== 10 pagos RECURRENTES con fecha de fin en el PASADO =====
    AltaPago(new Pago(MetodosPago.Credito, Usuarios[0],
TipoGastos[3], "Campaña marketing",new

```

```

InstanciaPagoRecurrente(12500, DateTime.Now.AddMonths(-14),
DateTime.Now.AddMonths(-6)));
    AltaPago(new Pago(MetodosPago.Debito, Usuarios[1],
TipoGastos[4], "Contrato limpieza",new
InstanciaPagoRecurrente(3800, DateTime.Now.AddMonths(-12),
DateTime.Now.AddMonths(-3)));
    AltaPago(new Pago(MetodosPago.Credito, Usuarios[2],
TipoGastos[5], "Consultoría RRHH",new
InstanciaPagoRecurrente(9200, DateTime.Now.AddMonths(-9),
DateTime.Now.AddMonths(-1)));
    AltaPago(new Pago(MetodosPago.Credito, Usuarios[3],
TipoGastos[0], "Capacitación personal",new
InstanciaPagoRecurrente(4600, DateTime.Now.AddMonths(-10),
DateTime.Now.AddMonths(-2)));
    AltaPago(new Pago(MetodosPago.Debito, Usuarios[4],
TipoGastos[1], "Servicio backup",new
InstanciaPagoRecurrente(1700, DateTime.Now.AddMonths(-11),
DateTime.Now.AddMonths(-5)));
    AltaPago(new Pago(MetodosPago.Credito, Usuarios[5],
TipoGastos[2], "Alquiler sala reuniones",new
InstanciaPagoRecurrente(3000, DateTime.Now.AddMonths(-7),
DateTime.Now.AddMonths(-4)));
    AltaPago(new Pago(MetodosPago.Credito, Usuarios[6],
TipoGastos[3], "Publicidad trimestral",new
InstanciaPagoRecurrente(8000, DateTime.Now.AddMonths(-8),
DateTime.Now.AddMonths(-2)));
    AltaPago(new Pago(MetodosPago.Debito, Usuarios[7],
TipoGastos[4], "Soporte ERP",new InstanciaPagoRecurrente(5100,
DateTime.Now.AddMonths(-15), DateTime.Now.AddMonths(-9)));
    AltaPago(new Pago(MetodosPago.Debito, Usuarios[8],
TipoGastos[5], "Servicio jardinería",new
InstanciaPagoRecurrente(2400, DateTime.Now.AddMonths(-13),
DateTime.Now.AddMonths(-8)));
    AltaPago(new Pago(MetodosPago.Credito, Usuarios[9],
TipoGastos[0], "Monitoreo cámaras",new
InstanciaPagoRecurrente(3500, DateTime.Now.AddMonths(-6),
DateTime.Now.AddMonths(-1)));

    // ===== 17 pagos ÚNICOS (con número de recibo agregado) =====
    AltaPago(new Pago(MetodosPago.Efectivo, Usuarios[10],
TipoGastos[1], "Compra insumos oficina",new
InstanciaPagoUnico(2700, DateTime.Now.AddDays(-25), 2001)));
    AltaPago(new Pago(MetodosPago.Debito, Usuarios[11],
TipoGastos[2], "Flete puntual mercadería",new
InstanciaPagoUnico(15400, DateTime.Now.AddDays(-10), 2002)));
    AltaPago(new Pago(MetodosPago.Efectivo, Usuarios[12],
TipoGastos[3], "Reparación impresora",new InstanciaPagoUnico(4300,
DateTime.Now.AddDays(-40), 2003)));

```

```
AltaPago(new Pago(MetodosPago.Efectivo, Usuarios[13],
TipoGastos[4], "Catering reunión clientes",new
InstanciaPagoUnico(9600, DateTime.Now.AddDays(-5), 2004)));
AltaPago(new Pago(MetodosPago.Debito, Usuarios[14],
TipoGastos[5], "Taxi traslado urgente",new InstanciaPagoUnico(850,
DateTime.Now.AddDays(-3), 2005)));
AltaPago(new Pago(MetodosPago.Efectivo, Usuarios[0],
TipoGastos[0], "Reposición sillas",new InstanciaPagoUnico(11800,
DateTime.Now.AddDays(-60), 2006)));
AltaPago(new Pago(MetodosPago.Efectivo, Usuarios[1],
TipoGastos[1], "Papelería y timbres",new InstanciaPagoUnico(2100,
DateTime.Now.AddDays(-15), 2007)));
AltaPago(new Pago(MetodosPago.Debito, Usuarios[2],
TipoGastos[2], "Carga combustible única",new
InstanciaPagoUnico(4200, DateTime.Now.AddDays(-7), 2008)));
AltaPago(new Pago(MetodosPago.Debito, Usuarios[3],
TipoGastos[3], "Compra luces LED",new InstanciaPagoUnico(7800,
DateTime.Now.AddDays(-20), 2009)));
AltaPago(new Pago(MetodosPago.Efectivo, Usuarios[4],
TipoGastos[4], "Pago peaje excepcional",new
InstanciaPagoUnico(340, DateTime.Now.AddDays(-1), 2010)));
AltaPago(new Pago(MetodosPago.Debito, Usuarios[5],
TipoGastos[5], "Servicio plomería",new InstanciaPagoUnico(5200,
DateTime.Now.AddDays(-13), 2011)));
AltaPago(new Pago(MetodosPago.Debito, Usuarios[6],
TipoGastos[0], "Compra monitor 27",new InstanciaPagoUnico(18500,
DateTime.Now.AddDays(-33), 2012)));
AltaPago(new Pago(MetodosPago.Efectivo, Usuarios[7],
TipoGastos[1], "Cafetera nueva",new InstanciaPagoUnico(4200,
DateTime.Now.AddDays(-18), 2013)));
AltaPago(new Pago(MetodosPago.Debito, Usuarios[8],
TipoGastos[2], "Reemplazo neumático",new InstanciaPagoUnico(9300,
DateTime.Now.AddDays(-22), 2014)));
AltaPago(new Pago(MetodosPago.Efectivo, Usuarios[9],
TipoGastos[3], "Servicio desinfección",new
InstanciaPagoUnico(6100, DateTime.Now.AddDays(-27), 2015)));
AltaPago(new Pago(MetodosPago.Efectivo, Usuarios[10],
TipoGastos[4], "Compra matafuegos",new InstanciaPagoUnico(3900,
DateTime.Now.AddDays(-45), 2016)));
AltaPago(new Pago(MetodosPago.Debito, Usuarios[11],
TipoGastos[5], "Reparación notebook",new InstanciaPagoUnico(7200,
DateTime.Now.AddDays(-12), 2017)));
}
```

3. Código fuente

3.1 Clase Sistema

```
using Clases.Pagos;
using Clases.Usuarios;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace ObligatorioP2
{
    public class Sistema
    {
        private List<Usuario> Usuarios;
        private List<Pago> Pagos;
        private List<Equipo> Equipos;
        private List<TipoGasto> TipoGastos;
        private static Sistema Instancia;

        public Sistema()
        {
            Usuarios = new List<Usuario>();
            Pagos = new List<Pago>();
            Equipos = new List<Equipo>();
            TipoGastos = new List<TipoGasto>();
            PrecargaDatos();
        }

        public static Sistema GetSistema()
        {
            if (Instancia == null)
            {
                Instancia = new Sistema();
            }
            return Instancia;
        }

        public List<Usuario> GetUsuarios()
        {
            return Usuarios;
        }

        public List<Pago> GetPagos()
        {
            return Pagos;
        }
    }
}
```

```

public List<Equipo> GetEquipos()
{
    return Equipos;
}
public List<TipoGasto> GetTipoGastos()
{
    return TipoGastos;
}
public void AltaUsuario(Usuario usuario)
{
    try
    {
        if (usuario == null)
        {
            throw new Exception("El usuario no puede ser
nulo.");
        }
        if (Usuarios.Contains(usuario))
        {
            throw new Exception("El usuario ya existe.");
        }
        usuario.Validar();
        usuario.SetEmail(GenerarEmail(usuario));
    }
    catch (Exception ex)
    {
        throw new Exception("Error al validar el usuario: "
+ ex.Message);
    }
    Usuarios.Add(usuario);
}
public void AltaEquipo(Equipo equipo)
{
    try
    {
        if (equipo == null)
        {
            throw new Exception("El equipo no puede ser
nulo.");
        }
        if (Equipos.Contains(equipo))
        {
            throw new Exception("El equipo ya existe.");
        }
        equipo.Validar();
    }
    catch (Exception ex)
    {

```

```

        throw new Exception("Error al validar el equipo: "
+ ex.Message);
    }
    Equipos.Add(equipo);
}
public void AltaTipoGasto(TipoGasto tipoGasto)
{
    try
    {
        if (tipoGasto == null)
        {
            throw new Exception("El tipo de gasto no puede
ser nulo.");
        }
        if (TipoGastos.Contains(tipoGasto))
        {
            throw new Exception("El tipo de gasto ya
existe.");
        }
        tipoGasto.Validar();
    }
    catch (Exception ex)
    {
        throw new Exception("Error al validar el tipo de
gasto: " + ex.Message);
    }
    TipoGastos.Add(tipoGasto);
}
public void AltaPago(Pago pago)
{
    try
    {
        if (pago == null)
        {
            throw new Exception("El pago no puede ser
nulo.");
        }
        if (Pagos.Contains(pago))
        {
            throw new Exception("El pago ya existe.");
        }
        pago.Validar();
    }
    catch (Exception ex)
    {
        throw new Exception("Error al validar el pago: " +
ex.Message);
    }
}

```

```

        Pagos.Add(pago);
    }
    public Usuario GetUsuarioPorEmail(string email)
    {
        foreach (Usuario usuario in Usuarios)
        {
            if (usuario.Email == email)
            {
                return usuario;
            }
        }
        return null;
    }

    public string GenerarEmail(Usuario usuario)
    {
        int counter = 0;
        string email = null;
        bool flag = false;

        while (!flag)
        {
            email = usuario.CreateEmail(counter);

            if (GetUsuarioPorEmail(email) == null)
            {
                flag = true;
            }
            counter++;
        }
        return email;
    }

    public List<Pago> PagosPorUsuario(Usuario user)
    {
        List<Pago> lisAux = new List<Pago>();
        foreach (Pago p in Pagos)
        {
            if (p.UsuarioAsociado.Email == user.Email)
            {
                lisAux.Add(p);
            }
        }
        return lisAux;
    }

    public TipoGasto GetTipoGasto(String nombreGasto)
    {
        foreach (TipoGasto tipo in TipoGastos)
    }

```



```

        {
            if (tipo.Nombre == nombreGasto)
            {
                return tipo;
            }
        }
        return null;
    }

    public Equipo GetEquipoPorNombre(string nombre)
    {
        foreach (Equipo equipo in Equipos)
        {
            if (equipo.Nombre.ToLower() == nombre.ToLower())
            {
                return equipo;
            }
        }
        return null;
    }

    public List<Usuario> GetUsuariosDeEquipo(string
nombreEquipo)
    {
        Equipo equipo = GetEquipoPorNombre(nombreEquipo);
        if (equipo == null)
        {
            throw new Exception("El equipo no existe.");
        }

        return equipo.GetMiembros();
    }

    public void PrecargaDatos()
    {
        PrecargaEquipo();
        PrecargaUsuarios();
        PrecargaGastos();
        PrecargaPagos();
    }

    public void PrecargaEquipo()
    {
        Equipo contabilidad = new Equipo("Contabilidad");
        AltaEquipo(contabilidad);
    }

```

```

        Equipo finanzas = new Equipo("Finanzas");
        AltaEquipo(finanzas);

        Equipo tesoreria = new Equipo("Tesoreria");
        AltaEquipo(tesoreria);

        Equipo auditoria = new Equipo("Auditoria");
        AltaEquipo(auditoria);

        Equipo ventas = new Equipo("Ventas");
        AltaEquipo(ventas);

    }

    public void PrecargaUsuarios()
    {
        // Equipos existentes
        Equipo contabilidad = GetEquipoPorNombre("Contabilidad");
        Equipo finanzas      = GetEquipoPorNombre("Finanzas");
        Equipo tesoreria      = GetEquipoPorNombre("Tesoreria");
        Equipo auditoria      = GetEquipoPorNombre("Auditoria");

        // ---- Contabilidad (6) ----
        Usuario usuario1 = new Usuario("Juana", "Melcer", "12345678",
contabilidad);
        AltaUsuario(usuario1);
        contabilidad.AgregarMiembro(usuario1);

        Usuario usuario2 = new Usuario("Ana", "Perez", "12345678",
contabilidad);
        AltaUsuario(usuario2);
        contabilidad.AgregarMiembro(usuario2);

        Usuario usuario3 = new Usuario("Lucas", "Garcia", "12345678",
contabilidad);
        AltaUsuario(usuario3);
        contabilidad.AgregarMiembro(usuario3);

        Usuario usuario4 = new Usuario("Martina", "Rodriguez",
"12345678", contabilidad);
        AltaUsuario(usuario4);
        contabilidad.AgregarMiembro(usuario4);

        Usuario usuario5 = new Usuario("Santiago", "Fernandez",
"12345678", contabilidad);
        AltaUsuario(usuario5);
        contabilidad.AgregarMiembro(usuario5);
    }
}

```

```
    Usuario usuario6 = new Usuario("Valentina", "Gomez",
"12345678", contabilidad);
    AltaUsuario(usuario6);
    contabilidad.AgregarMiembro(usuario6);

    // ---- Finanzas (5) ----
    Usuario usuario7 = new Usuario("Mateo", "Suarez", "12345678",
finanzas);
    AltaUsuario(usuario7);
    finanzas.AgregarMiembro(usuario7);

    Usuario usuario8 = new Usuario("Camila", "Lopez", "12345678",
finanzas);
    AltaUsuario(usuario8);
    finanzas.AgregarMiembro(usuario8);

    Usuario usuario9 = new Usuario("Joaquin", "Alvarez",
"12345678", finanzas);
    AltaUsuario(usuario9);
    finanzas.AgregarMiembro(usuario9);

    Usuario usuario10 = new Usuario("Sofia", "Martinez",
"12345678", finanzas);
    AltaUsuario(usuario10);
    finanzas.AgregarMiembro(usuario10);

    Usuario usuario11 = new Usuario("Tomas", "Pereira", "12345678",
finanzas);
    AltaUsuario(usuario11);
    finanzas.AgregarMiembro(usuario11);

    // ---- Tesoreria (5) ----
    Usuario usuario12 = new Usuario("Isabella", "Santos",
"12345678", tesoreria);
    AltaUsuario(usuario12);
    tesoreria.AgregarMiembro(usuario12);

    Usuario usuario13 = new Usuario("Agustin", "Cabrerera",
"12345678", tesoreria);
    AltaUsuario(usuario13);
    tesoreria.AgregarMiembro(usuario13);

    Usuario usuario14 = new Usuario("Mia", "Torres", "12345678",
tesoreria);
    AltaUsuario(usuario14);
    tesoreria.AgregarMiembro(usuario14);
```

```

        Usuario usuario15 = new Usuario("Facundo", "Silva", "12345678",
tesoreria);
        AltaUsuario(usuario15);
        tesoreria.AgregarMiembro(usuario15);

        Usuario usuario16 = new Usuario("Julia", "Ramos", "12345678",
tesoreria);
        AltaUsuario(usuario16);
        tesoreria.AgregarMiembro(usuario16);

        // ---- Auditoria (5) ----
        Usuario usuario17 = new Usuario("Nicolas", "Castro",
"12345678", auditoria);
        AltaUsuario(usuario17);
        auditoria.AgregarMiembro(usuario17);

        Usuario usuario18 = new Usuario("Florencia", "Vega",
"12345678", auditoria);
        AltaUsuario(usuario18);
        auditoria.AgregarMiembro(usuario18);

        Usuario usuario19 = new Usuario("Andres", "Morales",
"12345678", auditoria);
        AltaUsuario(usuario19);
        auditoria.AgregarMiembro(usuario19);

        Usuario usuario20 = new Usuario("Carolina", "Ruiz", "12345678",
auditoria);
        AltaUsuario(usuario20);
        auditoria.AgregarMiembro(usuario20);

        Usuario usuario21 = new Usuario("Bruno", "Herrera", "12345678",
auditoria);
        AltaUsuario(usuario21);
        auditoria.AgregarMiembro(usuario21);
        Usuario usuario22 = new Usuario("Lucia", "Acosta", "12345678",
auditoria);
        AltaUsuario(usuario22);
        auditoria.AgregarMiembro(usuario22);
    }

```

```

    public void PrecargaGastos()
    {
        TipoGasto tipo1 = new TipoGasto("Alquiler", "Pago
mensual por el alquiler de la oficina");
        AltaTipoGasto(tipo1);
    }

```

```

        TipoGasto tipo2 = new TipoGasto("Servicios", "Pago de
servicios como luz, agua, internet, etc.");
        AltaTipoGasto(tipo2);
        TipoGasto tipo3 = new TipoGasto("Sueldos", "Pago de
sueldos a los empleados");
        AltaTipoGasto(tipo3);
        TipoGasto tipo4 = new TipoGasto("Materiales de
oficina", "Compra de materiales de oficina como papel, bolígrafos,
etc.");
        AltaTipoGasto(tipo4);
        TipoGasto tipo5 = new TipoGasto("Publicidad", "Gastos
en publicidad y marketing");
        AltaTipoGasto(tipo5);
        TipoGasto tipo6 = new TipoGasto("Viajes", "Gastos en
viajes de negocios");
        AltaTipoGasto(tipo6);

    }

public void PrecargaPagos()
{
    // ===== 6 pagos RECURRENTES SIN fecha de fin (EndDate = null)
    =====
    AltaPago(new Pago(MetodosPago.Credito, Usuarios[0],
TipoGastos[0], "Alquiler mensual",new
InstanciaPagoRecurrente(28000, DateTime.Now.AddMonths(-12),
null)));
    AltaPago(new Pago(MetodosPago.Debito, Usuarios[1],
TipoGastos[1], "Servicios (UTE/ANTEL)",new
InstanciaPagoRecurrente(6500, DateTime.Now.AddMonths(-9),
null)));
    AltaPago(new Pago(MetodosPago.Efectivo, Usuarios[2],
TipoGastos[2], "Sueldos",new
InstanciaPagoRecurrente(120000,DateTime.Now.AddMonths(-10),
null)));
    AltaPago(new Pago(MetodosPago.Credito, Usuarios[3],
TipoGastos[3], "Mantenimiento",new InstanciaPagoRecurrente(4200,
DateTime.Now.AddMonths(-6), null)));
    AltaPago(new Pago(MetodosPago.Debito, Usuarios[4],
TipoGastos[4], "Internet",new InstanciaPagoRecurrente(1800,
DateTime.Now.AddMonths(-8), null)));
    AltaPago(new Pago(MetodosPago.Efectivo, Usuarios[5],
TipoGastos[5], "Limpieza",new InstanciaPagoRecurrente(3500,
DateTime.Now.AddMonths(-5), null)));

    // ===== 9 pagos RECURRENTES con fecha de fin en el FUTURO =====
    AltaPago(new Pago(MetodosPago.Credito, Usuarios[6],
TipoGastos[0], "Licencia software",new

```

```
InstanciaPagoRecurrente(2200, DateTime.Now.AddMonths(-4),
DateTime.Now.AddMonths(8)));
    AltaPago(new Pago(MetodosPago.Debito, Usuarios[7],
TipoGastos[1], "Seguro flota",new InstanciaPagoRecurrente(9500,
DateTime.Now.AddMonths(-7), DateTime.Now.AddMonths(5))));
    AltaPago(new Pago(MetodosPago.Efectivo, Usuarios[8],
TipoGastos[2], "Alquiler depósito",new
InstanciaPagoRecurrente(54000, DateTime.Now.AddMonths(-2),
DateTime.Now.AddMonths(10))));
    AltaPago(new Pago(MetodosPago.Credito, Usuarios[9],
TipoGastos[3], "Servicio guardia",new
InstanciaPagoRecurrente(4100, DateTime.Now.AddMonths(-3),
DateTime.Now.AddMonths(4))));
    AltaPago(new Pago(MetodosPago.Debito, Usuarios[10],
TipoGastos[4], "Telefonía móvil",new
InstanciaPagoRecurrente(2600, DateTime.Now.AddMonths(-1),
DateTime.Now.AddMonths(12))));
    AltaPago(new Pago(MetodosPago.Efectivo, Usuarios[11],
TipoGastos[5], "Mantenimiento HVAC",new
InstanciaPagoRecurrente(7800, DateTime.Now.AddMonths(-6),
DateTime.Now.AddMonths(2))));
    AltaPago(new Pago(MetodosPago.Credito, Usuarios[12],
TipoGastos[0], "Soporte técnico",new
InstanciaPagoRecurrente(3300, DateTime.Now.AddMonths(-5),
DateTime.Now.AddMonths(6))));
    AltaPago(new Pago(MetodosPago.Debito, Usuarios[13],
TipoGastos[1], "Licencias antivirus",new
InstanciaPagoRecurrente(1450, DateTime.Now.AddMonths(-8),
DateTime.Now.AddMonths(3))));
    AltaPago(new Pago(MetodosPago.Efectivo, Usuarios[14],
TipoGastos[2], "Estacionamiento",new
InstanciaPagoRecurrente(2100, DateTime.Now.AddMonths(-2),
DateTime.Now.AddMonths(7))));

    // ===== 10 pagos RECURRENTES con fecha de fin en el PASADO =====
    AltaPago(new Pago(MetodosPago.Credito, Usuarios[0],
TipoGastos[3], "Campaña marketing",new
InstanciaPagoRecurrente(12500, DateTime.Now.AddMonths(-14),
DateTime.Now.AddMonths(-6))));
    AltaPago(new Pago(MetodosPago.Debito, Usuarios[1],
TipoGastos[4], "Contrato limpieza",new
InstanciaPagoRecurrente(3800, DateTime.Now.AddMonths(-12),
DateTime.Now.AddMonths(-3))));
    AltaPago(new Pago(MetodosPago.Efectivo, Usuarios[2],
TipoGastos[5], "Consultoría RRHH",new
InstanciaPagoRecurrente(9200, DateTime.Now.AddMonths(-9),
DateTime.Now.AddMonths(-1))));
```

```

    AltaPago(new Pago(MetodosPago.Credito, Usuarios[3],
TipoGastos[0], "Capacitación personal",new
InstanciaPagoRecurrente(4600, DateTime.Now.AddMonths(-10),
DateTime.Now.AddMonths(-2))));
    AltaPago(new Pago(MetodosPago.Debito, Usuarios[4],
TipoGastos[1], "Servicio backup",new
InstanciaPagoRecurrente(1700, DateTime.Now.AddMonths(-11),
DateTime.Now.AddMonths(-5))));
    AltaPago(new Pago(MetodosPago.Efectivo, Usuarios[5],
TipoGastos[2], "Alquiler sala reuniones",new
InstanciaPagoRecurrente(3000, DateTime.Now.AddMonths(-7),
DateTime.Now.AddMonths(-4))));
    AltaPago(new Pago(MetodosPago.Credito, Usuarios[6],
TipoGastos[3], "Publicidad trimestral",new
InstanciaPagoRecurrente(8000, DateTime.Now.AddMonths(-8),
DateTime.Now.AddMonths(-2))));
    AltaPago(new Pago(MetodosPago.Debito, Usuarios[7],
TipoGastos[4], "Soporte ERP",new InstanciaPagoRecurrente(5100,
DateTime.Now.AddMonths(-15), DateTime.Now.AddMonths(-9))));
    AltaPago(new Pago(MetodosPago.Efectivo, Usuarios[8],
TipoGastos[5], "Servicio jardinería",new
InstanciaPagoRecurrente(2400, DateTime.Now.AddMonths(-13),
DateTime.Now.AddMonths(-8))));
    AltaPago(new Pago(MetodosPago.Credito, Usuarios[9],
TipoGastos[0], "Monitoreo cámaras",new
InstanciaPagoRecurrente(3500, DateTime.Now.AddMonths(-6),
DateTime.Now.AddMonths(-1))));

    // ===== 17 pagos ÚNICOS (con número de recibo agregado) =====
    AltaPago(new Pago(MetodosPago.Efectivo, Usuarios[10],
TipoGastos[1], "Compra insumos oficina",new
InstanciaPagoUnico(2700, DateTime.Now.AddDays(-25), 2001));
    AltaPago(new Pago(MetodosPago.Debito, Usuarios[11],
TipoGastos[2], "Flete puntual mercadería",new
InstanciaPagoUnico(15400, DateTime.Now.AddDays(-10), 2002));
    AltaPago(new Pago(MetodosPago.Credito, Usuarios[12],
TipoGastos[3], "Reparación impresora",new InstanciaPagoUnico(4300,
DateTime.Now.AddDays(-40), 2003));
    AltaPago(new Pago(MetodosPago.Efectivo, Usuarios[13],
TipoGastos[4], "Catering reunión clientes",new
InstanciaPagoUnico(9600, DateTime.Now.AddDays(-5), 2004));
    AltaPago(new Pago(MetodosPago.Debito, Usuarios[14],
TipoGastos[5], "Taxi traslado urgente",new InstanciaPagoUnico(850,
DateTime.Now.AddDays(-3), 2005));
    AltaPago(new Pago(MetodosPago.Credito, Usuarios[0],
TipoGastos[0], "Reposición sillas",new InstanciaPagoUnico(11800,
DateTime.Now.AddDays(-60), 2006));

```

```

    AltaPago(new Pago(MetodosPago.Efectivo, Usuarios[1],
TipoGastos[1], "Papelería y timbres",new InstanciaPagoUnico(2100,
DateTime.Now.AddDays(-15), 2007)));
    AltaPago(new Pago(MetodosPago.Debito, Usuarios[2],
TipoGastos[2], "Carga combustible única",new
InstanciaPagoUnico(4200, DateTime.Now.AddDays(-7), 2008)));
    AltaPago(new Pago(MetodosPago.Credito, Usuarios[3],
TipoGastos[3], "Compra luces LED",new InstanciaPagoUnico(7800,
DateTime.Now.AddDays(-20), 2009)));
    AltaPago(new Pago(MetodosPago.Efectivo, Usuarios[4],
TipoGastos[4], "Pago peaje excepcional",new
InstanciaPagoUnico(340, DateTime.Now.AddDays(-1), 2010)));
    AltaPago(new Pago(MetodosPago.Debito, Usuarios[5],
TipoGastos[5], "Servicio plomería",new InstanciaPagoUnico(5200,
DateTime.Now.AddDays(-13), 2011)));
    AltaPago(new Pago(MetodosPago.Credito, Usuarios[6],
TipoGastos[0], "Compra monitor 27",new InstanciaPagoUnico(18500,
DateTime.Now.AddDays(-33), 2012)));
    AltaPago(new Pago(MetodosPago.Efectivo, Usuarios[7],
TipoGastos[1], "Cafetera nueva",new InstanciaPagoUnico(4200,
DateTime.Now.AddDays(-18), 2013)));
    AltaPago(new Pago(MetodosPago.Debito, Usuarios[8],
TipoGastos[2], "Reemplazo neumático",new InstanciaPagoUnico(9300,
DateTime.Now.AddDays(-22), 2014)));
    AltaPago(new Pago(MetodosPago.Credito, Usuarios[9],
TipoGastos[3], "Servicio desinfección",new
InstanciaPagoUnico(6100, DateTime.Now.AddDays(-27), 2015)));
    AltaPago(new Pago(MetodosPago.Efectivo, Usuarios[10],
TipoGastos[4], "Compra matafuegos",new InstanciaPagoUnico(3900,
DateTime.Now.AddDays(-45), 2016)));
    AltaPago(new Pago(MetodosPago.Debito, Usuarios[11],
TipoGastos[5], "Reparación notebook",new InstanciaPagoUnico(7200,
DateTime.Now.AddDays(-12), 2017)));
}
}
}

```

3.2 Clase Usuario

```
using ObligatorioP2;
```



```

using System.Diagnostics.Metrics;

namespace Clases.Usuarios
{
    public class Usuario : IValidar
    {
        public int Id { get; set; }
        public static int UId { get; set; } = 0;
        public string Nombre { get; set; }
        public string Apellido { get; set; }
        public string Contrasenía { get; set; }
        public string Email { get; set; }

        public Equipo Equipo { get; set; }
        public DateTime FechaAlta { get; set; }
        public Usuario()
        {
            Id = UId++;
        }
        public Usuario(string nombre, string apellido, string
contrasenía, Equipo equipo)
        {
            Id = UId++;
            Nombre = nombre;
            Apellido = apellido;
            Contrasenía = contrasenía;
            Equipo = equipo;
            FechaAlta = DateTime.Now;
        }
        public override string ToString()
        {
            return $"Usuario: {Nombre} {Apellido}, Email: {Email},
Equipo: {Equipo.Nombre}";
        }
        public void Validar()
        {
            ValidarNombre();
            ValidarApellido();
            ValidarContrasenía();
            ValidarEquipo();
            ValidarFechaAlta();
        }
        private void ValidarNombre()
        {
            if (Nombre.Length < 3)
            {
                throw new Exception("El nombre debe tener al menos
3 caracteres.");
            }
        }
    }
}

```

```
    }  
    private void ValidarApellido()  
    {  
        if (Apellido.Length < 3)  
        {  
            throw new Exception("El apellido debe tener al  
menos 3 caracteres.");  
        }  
    }  
    private void ValidarContrasenia()  
    {  
        if (Contrasenia.Length < 8)  
        {  
            throw new Exception("La contraseña debe tener al  
menos 8 caracteres.");  
        }  
    }  
    private void ValidarEquipo()  
    {  
        if (Equipo == null)  
        {  
            throw new Exception("El equipo no Existe.");  
        }  
    }  
    private void ValidarFechaAlta()  
    {  
        if (FechaAlta == DateTime.MinValue)  
        {  
            throw new Exception("La fecha de alta no puede  
estar vacia.");  
        }  
    }  
    public string FirstThreeLetters(string str)  
    {  
        if (str.Length < 3)  
        {  
            return str;  
        }  
        return str.Substring(0, 3);  
    }  
    public string CreateEmail(int counter)  
    {  
        string email;  
        if (counter > 0)  
        {
```

```

        email = FirstThreeLetters(Nombre.ToLower()) +
FirstThreeLetters(Apellido.ToLower()) + counter +
"@laEmpresa.com";
        SetEmail(email);
        return email;
    }
    email = FirstThreeLetters(Nombre.ToLower()) +
FirstThreeLetters(Apellido.ToLower()) + "@laEmpresa.com";
    SetEmail(email);
    return email;
}
public void SetEmail(string email)
{
    ValidarEmail(email);
    Email = email;
}
public void ValidarEmail(string email)
{
    if (email.Length < 10)
    {
        throw new Exception("El email no es valido.");
    }
    if (!email.Contains("@") || !email.Contains("."))
    {
        throw new Exception("El email no es válido.");
    }
}

public string GetEmail()
{
    return Email;
}
public override bool Equals(object? obj)
{
    if (obj is Usuario)
    {
        Usuario usuario = (Usuario)obj;
        return Id == usuario.Id || Email == usuario.Email;
    }
    return false;
}

public string StringMiembrosEquipo()
{
    return $"Usuario: {Nombre} {Apellido}, Email: {Email}";
}

```

3.3 Clase Equipo

```
using ObligatorioP2;
using System.ComponentModel.DataAnnotations;

namespace Clases.Usuarios
{
    public class Equipo : IValidar
    {
        public int Id { get; set; }
        public static int UId { get; set; } = 0;
        public string Nombre { get; set; }
        private List<Usuario> Miembros { get; set; } = new
List<Usuario>();

        public Equipo()
        {
            Id = UId++;
        }

        public Equipo(string nombre)
        {
            Id = UId;
            Nombre = nombre;
            UId++;
        }

        public override string ToString()
        {
            return $"Id: {Id}, Nombre: {Nombre}";
        }

        public string GetNombre()
        {
            return Nombre;
        }

        public void Validar()
        {
            ValidarNombre();
        }

        private void ValidarNombre()
        {
            if(Nombre.Length < 3)
            {
```

```

        throw new Exception("El nombre del equipo debe
tener al menos 3 caracteres.");
    }
}

public List<Usuario> GetMiembros()
{
    return Miembros;
}

public void AgregarMiembro(Usuario usuario)
{
    if (usuario == null)
    {
        throw new Exception("El usuario no puede ser
nulo.");
    }
    if (Miembros.Contains(usuario))
    {
        throw new Exception("El usuario ya es miembro del
equipo.");
    }
    Miembros.Add(usuario);
}

public override bool Equals(object? obj)
{
    if (obj is Equipo)
    {
        Equipo equipo = (Equipo)obj;
        return Id == equipo.Id || Nombre == equipo.Nombre;
    }
    return false;
}
}
}

```

3.4 Clase Tipo de Gasto

```
using ObligatorioP2;

namespace Clases.Pagos
{
    public class TipoGasto : IValidar
    {
        public int Id { get; set; }
        public static int UId {get; set;} = 0;
        public string Nombre { get; set; }
        public string Descripcion { get; set; }

        public TipoGasto()
        {
            Id = UId++;
        }

        public TipoGasto(string nombre, string descripcion)
        {
            Id = UId++;
            Nombre = nombre;
            Descripcion = descripcion;
        }

        public void Validar()
        {
            ValidarNombre();
            ValidarDescripcion();
        }

        private void ValidarNombre()
        {
            if (string.IsNullOrEmpty(Nombre))
            {
                throw new Exception("El nombre no puede estar
vacío");
            }
        }

        private void ValidarDescripcion()
        {
            if (string.IsNullOrEmpty(Descripcion))
            {
                throw new Exception("La descripcion no puede estar
vacía");
            }
        }
    }
}
```

```

    }

    }

    public override string ToString()
    {
        return Nombre ;
    }

    public override bool Equals(object? obj)
    {
        if (obj is TipoGasto)
        {
            TipoGasto tipoGasto = (TipoGasto)obj;
            return Nombre == tipoGasto.Nombre || Id ==
tipoGasto.Id;
        }
        return false;
    }

    }
}

```

3.5 Clase Pago

```

using Clases.Usuarios;
using ObligatorioP2;

namespace Clases.Pagos
{
    public class Pago : IValidar
    {
        public int Id { get; set; }
        public static int UId { get; set; }
        public MetodosPago MetodosPago { get; set; }
        public Usuario UsuarioAsociado { get; set; }
        public TipoGasto TipoGasto { get; set; }
        public string Descripcion { get; set; }
        public InstanciaPago InstanciaPago { get; set; }
        public double MontoFinal { get; set; }

        public Pago()
        {
            Id = UId++;

```

```

    }

    public Pago(MetodosPago metodosPago, Usuario
usuarioAsociado, TipoGasto tipoGasto, string descripcion,
InstanciaPago instanciaPago)
    {
        Id = UId++;
        MetodosPago = metodosPago;
        UsuarioAsociado = usuarioAsociado;
        TipoGasto = tipoGasto;
        Descripcion = descripcion;
        InstanciaPago = instanciaPago;
        SetMontoFinal();
    }

    public void Validar()
    {
        ValidarDescripcion();
        ValidarMetodosPago();
        ValidarTipoGasto();
        ValidarInstanciaPago();
        ValidarMontoFinal();
        ValidarUsuarioAsociado();
    }
    private void ValidarUsuarioAsociado()
    {
        if (UsuarioAsociado == null)
        {
            throw new Exception("El usuario asociado no puede
ser nulo");
        }
    }
    private void ValidarMontoFinal()
    {
        if (MontoFinal <= 0)
        {
            throw new Exception("El monto final debe ser mayor
a 0");
        }
    }
    private void ValidarInstanciaPago()
    {
        if (InstanciaPago == null)
        {
            throw new Exception("La instancia de pago no puede
ser nula");
        }
        InstanciaPago.Validar();
    }

```



```

    }

    private void ValidarTipoGasto()
    {
        if (TipoGasto == null)
        {
            throw new Exception("El tipo de gasto no puede ser nulo");
        }
    }

    private double SetMontoFinal()
    {
        try
        {
            if (InstanciaPago == null)
            {
                throw new Exception("La instancia de pago no puede ser nula");
            }
            double montoFinal =
InstanciaPago.CalcularMontoPago(MetodosPago);
            MontoFinal = montoFinal;
            return MontoFinal;
        }
        catch (Exception ex)
        {
            throw new Exception("Error al calcular el monto final: " + ex.Message);
        }
    }

    private void ValidarMetodosPago()
    {
        if (MetodosPago != MetodosPago.Credito && MetodosPago != MetodosPago.Debito &&
MetodosPago != MetodosPago.Efectivo)
        {
            throw new Exception("Metodos pago invalido");
        }
    }

    private void ValidarDescripcion()
    {
        if (string.IsNullOrEmpty(Descripcion))
        {
            throw new Exception("La descripcion de puede estar vacia");
        }
    }

```

```

    }

    public override string ToString()
    {
        string texto = $"El Pago {Id} de monto {MontoFinal:F2}
fue abonado con el método \"{MetodosPago}\", por el usuario:
{(UsuarioAsociado != null ? UsuarioAsociado.ToString() :
"N/A")}. ";

        return texto + $" Detalles:
{InstanciaPago.ToString()}";
    }

    public override bool Equals(object? obj)
    {
        if (obj is Pago)
        {
            Pago pago = (Pago)obj;
            return Id == pago.Id;
        }
        return false;
    }
}

```

3.6 Clase Instancia Pago (Abstracta)

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using ObligatorioP2;

namespace Clases.Pagos
{
    public abstract class InstanciaPago : IValidar
    {
        public int Id { get; set; }
        public static int UId { get; set; } = 0;
        public double MontoBase { get; set; }

        public InstanciaPago()
        {

```

```
        Id = UId++;
    }
    public InstanciaPago(double montoBase)
    {
        Id = UId;
        MontoBase = montoBase;
        UId++;
    }

    public double GetMontoBase()
    {
        return MontoBase;
    }

    public abstract double CalcularMontoPago(MetodosPago
metodoPago);

    public abstract bool EsPagoActivo(DateTime mes);

    public abstract string MiTipo();

    public virtual void Validar()
    {
        ValidarMontoBase();
    }

    private void ValidarMontoBase()
    {
        if (MontoBase <= 0)
        {
            throw new Exception("El monto base debe ser mayor a
0.");
        }
    }
}
}
```

3.7 Clase Pago Recurrente

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Clases.Pagos
{
    public class InstanciaPagoRecurrente : InstanciaPago
    {
        private DateTime FechaInicio { get; set; }
        private DateTime? FechaFin { get; set; }

        public InstanciaPagoRecurrente(double montoBase, DateTime
fechaInicio, DateTime? fechaFin = null) : base(montoBase)
        {
            FechaInicio = fechaInicio;
            if (fechaFin != null)
            {
                FechaFin = fechaFin;
            }
        }

        public DateTime GetFechaInicio()
        {
            return FechaInicio;
        }

        public DateTime? GetFechaFin()
        {
            return FechaFin;
        }

        public override void Validar()
        {
            base.Validar();
            ValidarFechaInicio();
            ValidarFechaFin();
        }

        public void ValidarFechaInicio()
        {

```

```
        if (FechaInicio == DateTime.MinValue)
        {
            throw new Exception("El campo fecha de inicio no puede estar vacio.");
        }
    }

    public void ValidarFechaFin()
    {
        DateTime fechaInicio = GetFechaInicio();
        if (FechaFin < fechaInicio)
        {
            throw new Exception("La fecha fin no puede ser anterior a la fecha de inicio.");
        }
    }

    private double CalcularRecargo()
    {
        int cuotasRestantes = GetCuotas();
        double recargo;
        switch (cuotasRestantes)
        {
            case <= 5:
                recargo = 1.03;
                break;
            case >= 10:
                recargo = 1.10;
                break;
            case >= 6 and <= 9:
                recargo = 1.05;
                break;
        }
        return recargo;
    }

    public override double CalcularMontoPago(MetodosPago metodoPago)
    {
        double montoBase = MontoBase;
        int cuotasRestantes = GetCuotas();
        double recargo = CalcularRecargo();

        if (cuotasRestantes == -1)
        {
            return montoBase * recargo;
        }
        else
```

```

        {
            return montoBase * cuotasRestantes * recargo;
        }
    }

    public int GetCuotas()
    {
        if (FechaFin == null)
        {
            return -1; // Indica que el pago es indefinido
        }
        DateTime fechaFin = FechaFin ?? DateTime.Now;
        int meses = (fechaFin.Year - FechaInicio.Year) * 12 +
fechaFin.Month - FechaInicio.Month;
        return meses + 1; // +1 para incluir el mes de inicio
    }

    public int CuotasRestantes()
    {
        if (FechaFin == null)
        {
            return -1; // Indica que el pago es indefinido
        }
        DateTime fechaFin = FechaFin ?? DateTime.Now;
        DateTime mesActual = DateTime.Now;

        int meses = (fechaFin.Year - mesActual.Year) * 12 +
fechaFin.Month - mesActual.Month;
        return meses + 1;
    }

    public override bool EsPagoActivo(DateTime mes)
    {
        return mes >= FechaInicio && (FechaFin == null || mes
<= FechaFin);
    }

    public override string MiTipo()
    {
        return "Recurrente";
    }

    public override string ToString()
    {
        int mesesRestantes = CuotasRestantes();
        if (mesesRestantes < 0)
        {
            return $"Recurrente";
        }
        return $"Cuotas restantes {mesesRestantes}";
    }
}
}

```

3.8 Clase Pago Único

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Clases.Pagos
{
    public class InstanciaPagoUnico : InstanciaPago
    {
        private DateTime FechaPago { get; set; }
        private int ReciboPago { get; set; }
        public InstanciaPagoUnico(double montoBase, DateTime
fechaPago, int reciboPago) : base(montoBase)
        {
            FechaPago = fechaPago;
            ReciboPago = reciboPago;
        }
        public override void Validar()
        {
            base.Validar();
            ValidarFechaPago();
            ValidarReciboPago();
        }
        public DateTime GetFechaPago()
        {
            return FechaPago;
        }
        public int GetReciboPago()
        {
            return ReciboPago;
        }
        private void ValidarFechaPago()
        {
            if (FechaPago == DateTime.MinValue)
            {
                throw new Exception("El campo fecha de pago no
puede estar vacio.");
            }
        }
        private void ValidarReciboPago()
```

```
        {
            if (ReciboPago <= 0)
            {
                throw new Exception("El numero de recibo debe ser
mayor a 0.");
            }
        }

        public override double CalcularMontoPago(MetodosPago
metodoPago)
        {

            if(metodoPago == MetodosPago.Efectivo)
            {
                return MontoBase * 0.8;
            }
            else
            {
                return MontoBase * 0.9;
            }
        }

        public override bool EsPagoActivo(DateTime mes)
        {
            return FechaPago.Month == mes.Month && FechaPago.Year
== mes.Year;
        }
        public override string MiTipo()
        {
            return "Unico";
        }
        public override string ToString()
        {
            return "Pago Unico";
        }
    }
}
```


3.9 Método Pago (Enumeration)

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Clases.Pagos
{
    public enum MetodosPago
    {
        Credito = 1,
        Debito = 2,
        Efectivo = 3
    }
}
```

3.10 IValidar (Interface)

```
namespace ObligatorioP2;

public interface IValidar
{
    public void Validar();
}
```

3.11 Program

[illegible]

```

        {
            Console.WriteLine("Por favor,
ingrese '1' para volver al menú.");
        }
        Console.Clear();
    }
    catch (Exception e)
    {
        Console.WriteLine(e.Message);
    }
    break;
case 2:
    ListarPagosPorEmail(sistema);
    break;
case 3:
    CrearUsuario(sistema);

    break;
case 4:
    try
    {
        Equipo equipo =
ListarEquiposParaSelec(sistema, "Seleccione el nombre del
equipo:");

        if (equipo != null)
        {
            List<Usuario> listaUsuariosDeEquipo
= sistema.GetUsuariosDeEquipo(equipo.GetNombre());
            if (listaUsuariosDeEquipo.Count ==
0)

            {
                Console.WriteLine("El equipo
ingresado no contiene usuarios");
                Console.WriteLine("");
                Console.WriteLine("1 - Volver
al menu ");
            }
            else
            {
                foreach (Usuario u in
listaUsuariosDeEquipo)

                {
                    Console.WriteLine(u.StringMiembrosEquipo());
                }
            }
        }
    }
}

```

```

        else { Console.WriteLine("Debe
seleccionar un equipo valido."); }
    }
    catch (Exception e)
    {

        Console.WriteLine(e.Message);
        Console.WriteLine("");
        Console.WriteLine("Por favor,
ingrese '1' para volver al menú.");
    }
    Console.WriteLine("1 - Volver al menu ");

    while (Console.ReadLine() != "1")
    {
        Console.WriteLine("Por favor,
ingrese '1' para volver al menú.");
    }
    Console.Clear();

    break;
default:
    Console.WriteLine("Por favor seleccione una
opcion correcta.");
    break;
}
}
Console.ReadKey();

}

private static void ListarPagos(List<Pago> listPagos)
{
    if (listPagos.Count > 0)
    {
        foreach (Pago p in listPagos)
        {
            Console.WriteLine(p.ToString());
        }
        Console.WriteLine("Fin de la lista.");
    }
    else
    {
        Console.WriteLine("No hay pagos en el sistema.");
    }
    Console.WriteLine("");
    Console.WriteLine("1 - Volver al menu ");
}

```

```

    }

    private static void ListarUsuarios(List<Usuario>
listaUsuarios)
    {
        if (listaUsuarios.Count > 0)
        {
            foreach (Usuario u in listaUsuarios)
            {
                Console.WriteLine(u.ToString());
            }
            Console.WriteLine("Fin de la lista.");
        }
        else
        {
            Console.WriteLine("No hay usuarios en el
sistema.");
        }
        Console.WriteLine("");
        Console.WriteLine("1 - Volver al menu ");
    }

    public static void MostrarMenu()
    {
        Console.Clear();
        Console.WriteLine("-----Seleccione una
opcion:-----");

        Console.WriteLine("-----");
        Console.WriteLine("1. Listar todos los Usuarios del
Sistema.");

        Console.WriteLine("-----");
        Console.WriteLine("2. Listar pagos de usuario por
correo.");

        Console.WriteLine("-----");
        Console.WriteLine("3. Alta de usuario");

        Console.WriteLine("-----");
        Console.WriteLine("4. Listar los miembros de un
Equipo");

        Console.WriteLine("-----");
        Console.WriteLine("0. Salir");
    }

    private static void ListarPagosPorEmail(Sistema sistema)

```

```

    {
        Console.WriteLine("Ingrese el email del Usuario:");
        string? email = Console.ReadLine();
        if (string.IsNullOrEmpty(email))
        {
            Console.WriteLine("El email no puede estar
vacío.");
            return;
        }
        if (!email.Contains("@") || !email.Contains("."))
        {
            Console.WriteLine("El email ingresado no es
válido.");
            return;
        }
        if (email.Length < 5)
        {
            Console.WriteLine("El email debe tener al menos 5
caracteres.");
            return;
        }
        if (!email.Contains("laEmpresa"))
        {
            Console.WriteLine("El email debe pertenecer a la
empresa (debe contener 'laEmpresa').");
            return;
        }
        Usuario usuario = sistema.GetUsuarioPorEmail(email);
        if (usuario != null)
        {
            try
            {
                List<Pago> listaPagos =
sistema.PagosPorUsuario(usuario);
                ListarPagos(listaPagos);
                // Espera hasta que el usuario escriba "1" para
volver al menú
                while (Console.ReadLine() != "1")
                {
                    Console.WriteLine("Por favor, ingrese '1'
para volver al menú.");
                }
                Console.Clear();
            }
            catch (Exception e)
            {
                Console.WriteLine(e.Message);
            }
        }
    }

```

```

    }
    else
    {
        Console.WriteLine("No existe un usuario con ese
email.");
    }
}
// Método para crear un usuario nuevo y agregarlo al sistema
private static void CrearUsuario(Sistema sistema)
{
    string nombre = "";
    while (true)
    {
        Console.WriteLine("Ingrese Nombre:");
        string input = Console.ReadLine().Trim();
        if (!string.IsNullOrEmpty(input) && input.Length >=
3)
        {
            nombre = input;
            break;
        }
        Console.WriteLine("El nombre debe tener al menos 3
caracteres.");
    }

    string apellido = "";
    while (true)
    {
        Console.WriteLine("Ingrese Apellido:");
        string input = Console.ReadLine().Trim();
        if (!string.IsNullOrEmpty(input) && input.Length >=
3)
        {
            apellido = input;
            break;
        }
        Console.WriteLine("El apellido debe tener al menos
3 caracteres.");
    }

    string contrasenia = "";
    while (true)
    {
        Console.WriteLine("Ingrese Contraseña:");
        string input = Console.ReadLine().Trim();
        if (!string.IsNullOrEmpty(input) && input.Length >=
8)
        {

```

```

        contrasenia = input;
        break;
    }
    Console.WriteLine("La contraseña debe tener al
menos 8 caracteres.");
}

Equipo equipo = null;
while (equipo == null)
{
    equipo = ListarEquiposParaSelec(sistema,
"Seleccione un Equipo:");
    if (equipo == null)
    {
        Console.WriteLine("Debe seleccionar un equipo
válido.");
    }
}

try
{
    Usuario usuNuevo = new Usuario(nombre, apellido,
contrasenia, equipo);
    sistema.AltaUsuario(usuNuevo);
    equipo.AgregarMiembro(usuNuevo);
    Console.WriteLine("Usuario creado correctamente.");
}
catch (Exception e)
{
    Console.WriteLine("Error: " + e.Message);
}
Console.WriteLine();
Console.WriteLine("1 - Volver al menú");
while (Console.ReadLine() != "1")
{
    Console.WriteLine("Por favor, ingrese '1' para
volver al menú.");
}
Console.Clear();
}

// Método para seleccionar un equipo de la lista de equipos
del sistema
private static Equipo? ListarEquiposParaSelec(Sistema
sistema, string text)
{
    List<Equipo> listaEquipos = sistema.GetEquipos();
    int counter = 0;
    Console.WriteLine(text);

```



```
        foreach (Equipo e in listaEquipos)
        {
            Console.WriteLine($"{counter++} - {e.GetNombre()}");
        }

        string? input = Console.ReadLine();
        if (!int.TryParse(input, out int indiceEquipo) ||
            indiceEquipo < 0 || indiceEquipo >= listaEquipos.Count)
        {
            Console.WriteLine("Opción de equipo inválida.");
            return null;
        }
        return listaEquipos[indiceEquipo];
    }
}
```

4. Prompts AI

Prompts utilizado para la precarga de datos >

Precarga Pagos: [Ver enlace](#)

Precarga Usuarios y Equipos: [Ver enlace](#)